SkyRoute - AI-Based Smart Drone Delivery Optimization System

SkyRoute simulates a fleet of autonomous delivery drones operating in a city grid, making real-time routing and dispatch decisions using classic AI search and constraint-based reasoning.

Built as part of coursework for Computational Foundations for AI, mapped to core AI concepts: informed search, constraint satisfaction, and agent-based decision making.

What It Does

Two drones operate across a simulated city, picking up and delivering Medicine, Food, and Shopping requests under real-world constraints - battery life, package weight, delivery time windows, and live weather conditions.

Every incoming delivery request goes through an AI decision pipeline that:

1. Runs **A* search** to find the shortest safe route between pickup and drop-off
2. Checks constraints = battery cost, package weight, delivery time window
3. Scores weather risk along the route (rain, wind, storm, fog all affect routing)
4. Selects the best available drone, or merges deliveries in eco-mode when possible
5. Accepts, queues, or rejects the request based on feasibility

Features

- **A* pathfinding** across a 30×22 grid city with buildings, no-fly zones, and charging stations
- **Dynamic weather system** — rain, wind, storm, and fog probabilities shift over time and increase route risk and battery drain
- **Priority-based delivery queue** — emergency medicine requests are prioritized over food and shopping
- **Battery management** — drones estimate battery cost per route and autonomously navigate to charging stations when low
- **Eco-merge logic** — combines compatible deliveries onto a single drone trip when efficient
- **Live sidebar dashboard** — drone status, battery levels, active weather, and delivery stats rendered in real time

How It Works

Module	Responsibility
`main.py`	Simulation loop, rendering, controls
`drone.py`	Drone agent — movement, battery, state machine
`pathfinding.py`	A* search algorithm and battery cost estimation
`queue_manager.py`	Delivery decision pipeline — accept / reject / queue / eco-merge
`delivery.py`	Delivery data model and request generator
`weather.py`	Probabilistic weather system affecting routing risk
`map.py`	City grid — buildings, no-fly zones, charging/pickup/drop-off points
`constants.py`	Centralized configuration

Controls

Key	Action
`SPACE`	Start / unpause simulation
`E`	Send Emergency Medicine delivery
`F`	Send Food delivery
`S`	Send Shopping delivery
`Q`	Quit

Tech Stack

Python, Pygame

Run Locally

pip install pygame
python main.py